ELEC 475: Lab 2 — Post-Lab Manual

Deepening the Network: Full AlexNet, Normalization Study, & Data Augmentation

Course: ELEC 475 — Computer Vision with Deep Learning (Fall 2026)
Department: Electrical & Computer Engineering, Queen's University
Instructor: Dr. Michael Greenspan
Lab Developer & Teaching Assistant: Arshia Esmail Tehrani (`hd53@queensu.ca`)
Teaching Team: Yawar Ashraf, Connor Johnston, Tyler Wilson, Naser Al-Obeidat
Due Date: October 22, 2026 (before 23:59 ET) per Course Syllabus & Master Schedule
Weight: 5 Marks (In-Lab 3 Marks + Post-Lab 5 Marks = 8 Marks Total, 7% Course Weight)

Table of Contents

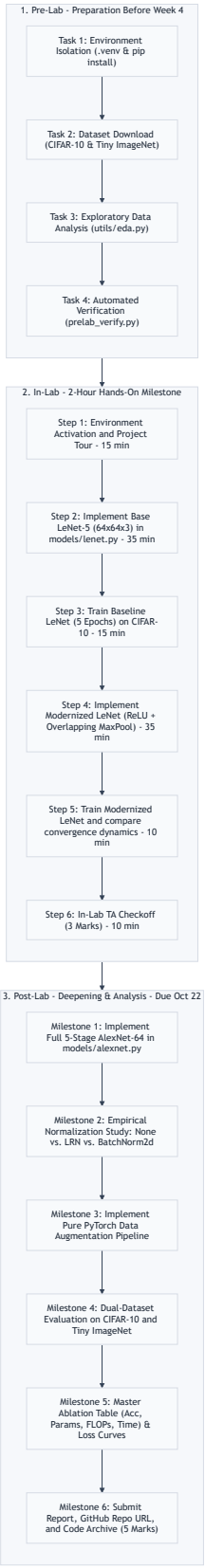
1. The 3-Phase Lab 2 Agenda
2. Post-Lab Overview & Objectives
3. Step 1: Implement Full AlexNet for 64×64 Input
    - 3.1 Architecture Specification & Vertical Schematic Diagram
    - 3.2 Architectural Details & Spatial Math
    - 3.3 Parameter Verification
4. Step 2: Empirical Normalization Study (None vs. LRN vs. BatchNorm2d)
    - 4.1 Theoretical Background: The Transition from LRN to BatchNorm
    - 4.2 Layer Comparison Diagram (Vertical)
    - 4.3 Training the Three Normalization Variants
5. Step 3: PyTorch Data Augmentation Pipeline
    - 5.1 Augmentation Implementation in transforms.py
    - 5.2 Measuring the Generalization Gap
6. Step 4: Dual-Dataset Benchmark (Tiny ImageNet Challenge)
7. Step 5: Master Ablation Study Summary Tables, Exploration Menu & Model Profiling
    - 7.1 Running the Automated Architecture Benchmarking Harness
    - 7.2 Recording Training Wall-Clock Time
    - 7.3 Master Ablation Study Summary Tables
8. Step 6: Post-Lab Analytical Report Guidelines

    - 8.1 Required Report Sections

9. Step 7: Submission Guidelines & Rubric
    - 9.1 Submission Package Format & GitHub Repository Verification
    - 9.2 Lab 2 Overall Marking Rubric (7 Marks Total)

---

1. The 3-Phase Lab 2 Agenda



2. Post-Lab Overview & Objectives

In the In-Lab session, you confirmed that transitioning from Tanh to ReLU and from Average Pooling to Overlapping Max Pooling yielded a substantial validation accuracy boost and accelerated convergence on LeNet.

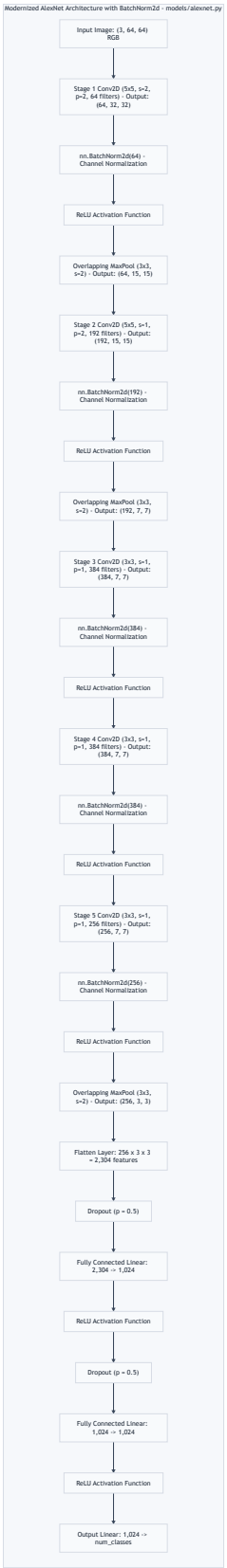
In this Post-Lab, you will complete the full architectural journey to **AlexNet (Krizhevsky et al., 2012)** by:

1. Building a 5-stage deep convolutional feature hierarchy with large channel capacity.
2. Conducting an empirical normalization ablation study: comparing **No Normalization vs. Local Response Normalization (LRN) vs. Batch Normalization (`BatchNorm2d`)**.
3. Implementing a pure PyTorch GPU **Data Augmentation Pipeline** (`RandomCrop`, `RandomHorizontalFlip`, `ColorJitter`) to combat overfitting.
4. Evaluating performance across both **CIFAR-10** and the challenging **Tiny ImageNet-200** dataset.
5. Providing your active **GitHub Repository URL** demonstrating clean branching and commit history.
6. Authoring a formal engineering report synthesizing your empirical findings.

Step 1: Implement Full AlexNet for 64×64 Input

Open `models/alexnet.py`. You will implement the complete `AlexNet64` class adapted for \(64 \times 64 \times 3\) input images.

3.1 Architecture Specification & Vertical Schematic Diagram



3.2 Architectural Details & Spatial Math:

1. **Stage 1:** `Conv2d(3, 64, kernel_size=5, stride=2, padding=2)` \(\implies (64, 32, 32) \rightarrow\) Norm \(\rightarrow\) ReLU \(\rightarrow\) `MaxPool2d(3, stride=2)` \(\implies (64, 15, 15)\).
2. **Stage 2:** `Conv2d(64, 192, kernel_size=5, stride=1, padding=2)` \(\implies (192, 15, 15) \rightarrow\) Norm \(\rightarrow\) ReLU \(\rightarrow\) `MaxPool2d(3, stride=2)` \(\implies (192, 7, 7)\).
3. **Stage 3:** `Conv2d(192, 384, kernel_size=3, stride=1, padding=1)` \(\implies (384, 7, 7) \rightarrow\) Norm \(\rightarrow\) ReLU.
4. **Stage 4:** `Conv2d(384, 384, kernel_size=3, stride=1, padding=1)` \(\implies (384, 7, 7) \rightarrow\) Norm \(\rightarrow\) ReLU.
5. **Stage 5:** `Conv2d(384, 256, kernel_size=3, stride=1, padding=1)` \(\implies (256, 7, 7) \rightarrow\) Norm \(\rightarrow\) ReLU \(\rightarrow\) `MaxPool2d(3, stride=2)` \(\implies (256, 3, 3)\).
6. **Classifier Head:**
    - Flatten: \(256 \times 3 \times 3 = \mathbf{2,304}\) features.
    - `Dropout(p=0.5)` \(\rightarrow\) `Linear(2304, 1024)` \(\rightarrow\) `ReLU()`.
    - `Dropout(p=0.5)` \(\rightarrow\) `Linear(1024, 1024)` \(\rightarrow\) `ReLU()`.
    - `Linear(1024, num_classes)`.

3.3 Parameter Verification

Verify your model parameters:

```
python -m models.alexnet
```

Ensure total trainable parameters equal \(\approx \mathbf{6.3\text{M}}\) parameters.

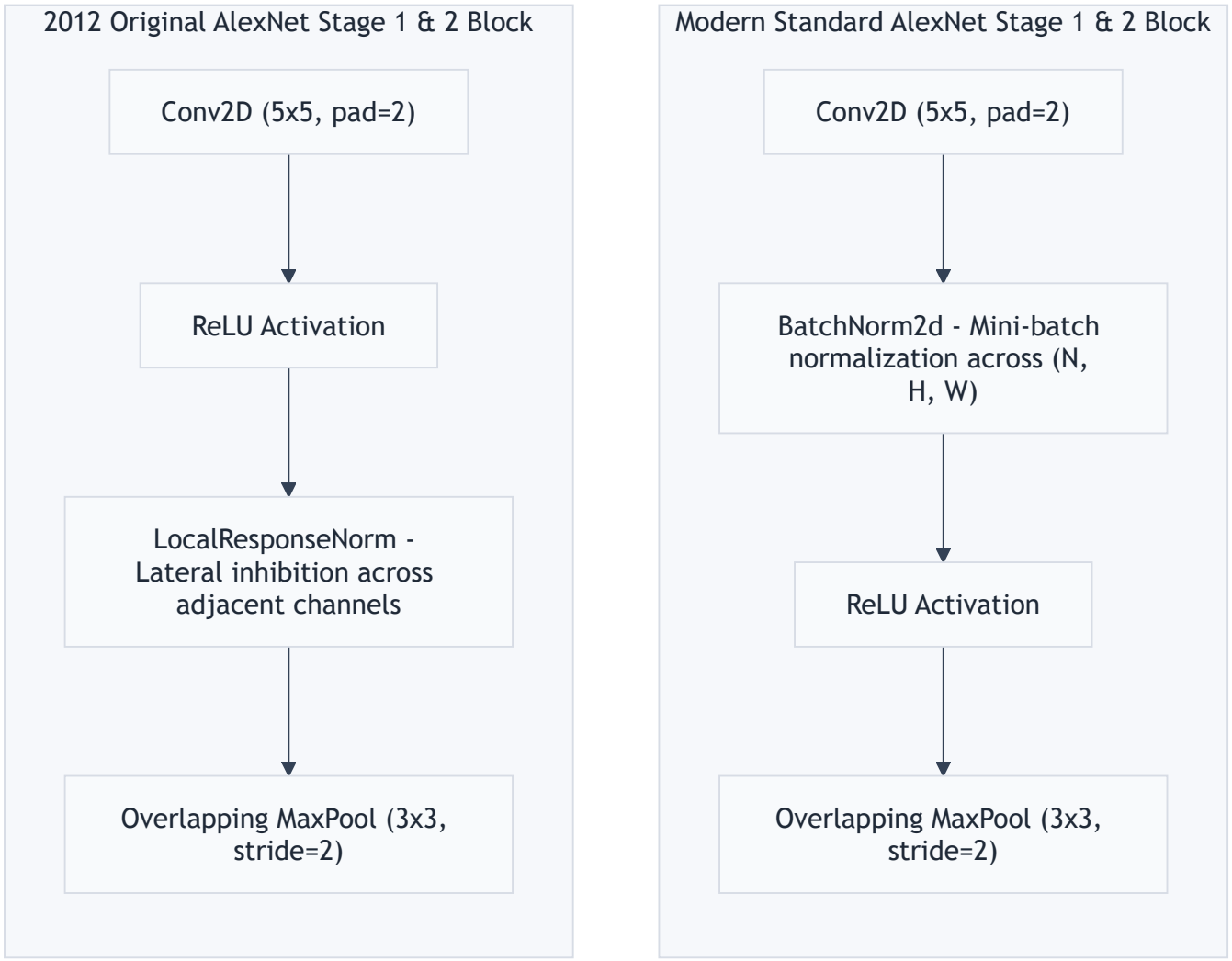
Step 2: Empirical Normalization Study (None vs. LRN vs. BatchNorm2d)

4.1 Theoretical Background: The Transition from LRN to BatchNorm

In the original 2012 AlexNet paper, Krizhevsky et al. introduced **Local Response Normalization (LRN)** to mimic biological lateral inhibition (where excited neurons dampen nearby neurons). However, subsequent computer vision architectures (such as VGG in 2014) demonstrated that LRN introduced computational overhead with minimal accuracy gain. In 2015, Ioffe and Szegedy introduced **Batch Normalization (`BatchNorm2d`)**, which normalizes feature distributions across the mini-batch dimension, fundamentally stabilizing training and accelerating convergence.

In this step, you will implement both normalization schemes and empirically measure their effect on training stability and convergence.

4.2 Layer Comparison Diagram (Vertical)



4.3 Training the Three Normalization Variants

Train three separate variants of AlexNet for 10 epochs each on CIFAR-10:

1. **Variant A — No Normalization (`--norm none`):**

```
python train.py --model alexnet --norm none --dataset cifar10 --epochs 10 --save-model checkpoints/alexnet_none.pth --plot-file plots/loss_alexnet_none.png
```

2. **Variant B — Local Response Normalization (`--norm lrn`):** Uses `nn.LocalResponseNorm(size=5, alpha=1e-4, beta=0.75, k=2.0)`.

```
python train.py --model alexnet --norm lrn --dataset cifar10 --epochs 10 --save-model checkpoints/alexnet_lrn.pth --plot-file plots/loss_alexnet_lrn.png
```

3. **Variant C — Modern Batch Normalization (`--norm bn`):** Uses `nn.BatchNorm2d(num_features)`.

```
python train.py --model alexnet --norm bn --dataset cifar10 --epochs 10 --save-model checkpoints/alexnet_bn.pth --plot-file plots/loss_alexnet_bn.png
```

Empirical Observations & Questions to Address in Your Report:

- **Performance Improvement:** Did incorporating Batch Normalization yield an improvement in validation accuracy and Macro F_1 over No Normalization and LRN? (**Yes / No**).
    - **If Yes:** Explain the underlying mathematical and optimization reasons. How does normalizing mini-batch activations reduce internal covariate shift and smooth the optimization landscape?
    - **If No or Marginal:** Discuss what experimental parameters (e.g., choice of learning rate, batch size, or short 10-epoch horizon) may have limited the observable gains.
- **Computational Overhead:** Did the addition of LRN or `BatchNorm2d` measurably alter your training epoch wall-clock time and arithmetic FLOPs? (**Yes / No**). Contrast the computational overhead of element-wise normalization against the convergence benefit.
- **Convergence Stability:** Compare the variance and smoothness of the training loss trajectories across the three variants. Which normalization scheme demonstrated the most stable gradient progression?

Step 3: PyTorch Data Augmentation Pipeline

As deep neural networks increase in capacity (scaling from $\sim 0.34\text{ M}$ parameters in LeNet to $>6.3\text{ M}$ in AlexNet), they become prone to **overfitting** (memorizing training images without learning invariant representations).

5.1 Augmentation Implementation in `utils/transforms.py`

In `utils/transforms.py`, implement the training augmentation pipeline using pure PyTorch:

```
def get_train_transforms(dataset_name="cifar10"):
    return transforms.Compose([
        transforms.Resize((64, 64)),
        transforms.RandomCrop(64, padding=4, padding_mode="reflect"),
        transforms.RandomHorizontalFlip(p=0.5),
        transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.4914, 0.4822, 0.4465],
                             std=[0.2470, 0.2435, 0.2616])
    ])
```

Train AlexNet (with BatchNorm and Dropout) with and without data augmentation for 15 epochs:

```
# Without Data Augmentation:
python train.py --model alexnet --norm bn --dataset cifar10 --epochs 15 --no-augment --save-
model checkpoints/alexnet_no_aug.pth --plot-file plots/loss_no_aug.png

# With Pure PyTorch Data Augmentation:
python train.py --model alexnet --norm bn --dataset cifar10 --epochs 15 --save-model
checkpoints/alexnet_with_aug.pth --plot-file plots/loss_with_aug.png
```

5.2 Measuring the Generalization Gap

Calculate the **Generalization Gap**: $\Delta = \text{Training Accuracy} - \text{Validation Accuracy}$

Empirical Questions to Address in Your Report:

- **Generalization Gap Reduction:** Did applying the PyTorch data augmentation pipeline reduce the generalization gap Δ? (**Yes / No**).
    - Report your measured training accuracy, validation accuracy, and calculated Δ for both the unaugmented run and the augmented run.
- **Validation Metric Improvement:** Did data augmentation lead to an overall increase in final validation accuracy and Macro F_1? (**Yes / No**).
    - **If Yes:** Explain why injecting label-preserving stochastic transformations (random crops, horizontal flips, color jitter) forces convolutional filters to learn invariant semantic features rather than memorizing canonical training sample positions.
    - **If No or Marginal:** Inspect your training curves. Did the network require more epochs to fully assimilate the augmented distribution, or did severe transformations cause underfitting?

Step 4: Dual-Dataset Benchmark (Tiny ImageNet Challenge)

Now evaluate your AlexNet on the **Tiny ImageNet-200** dataset (or the 10-class benchmark subset):

```
python train.py --model alexnet --norm bn --dataset tiny_imagenet --epochs 10 --save-model checkpoints/alexnet_tiny_imagenet.pth --plot-file plots/loss_tiny_imagenet.png
```

Empirical Questions to Address in Your Report:

- **Generalization Above Random Chance:** Did AlexNet learn discriminative features on Tiny ImageNet significantly above the random guess baseline (\(1/200 = 0.5\%\))? (**Yes / No**).
- **Cross-Dataset Performance Comparison:** How do your validation accuracy and Macro \(F_1\) on Tiny ImageNet compare to CIFAR-10? Why does Tiny ImageNet present a vastly steeper generalization challenge (considering 200 fine-grained categories, \(64\times 64\) spatial complexity, and reduced training samples per class)?
- **Overfitting Trajectory:** Did the Tiny ImageNet validation loss diverge or plateau earlier in the training schedule compared to CIFAR-10? Explain this behavior using your generated loss curves.

Step 5: Master Ablation Study Summary Tables, Exploration Menu & Model Profiling

To complete your empirical evaluation, you must benchmark all models across both **Predictive Performance** and **Computational Complexity**, matching the academic profiling standards established in foundational computer vision literature (He et al., CVPR 2016; Howard et al., MobileNets 2017).

7.1 Running the Automated Architecture Benchmarking Harness

Run the provided benchmarking harness to compute exact Parameters, MACs, FLOPs, Latency, and Throughput across all models:

```
python compare_models.py --dataset cifar10 --device cuda
```

7.2 Recording Training Wall-Clock Time

In `train.py`, each epoch is automatically wrapped in our high-precision `Timer` (with CUDA synchronization). Notice the epoch summary printout:

```
Epoch 01/10 Summary: Train Loss: 1.4520, Train Acc: 48.20% | Val Loss: 1.2840, Val Acc: 54.10% | Time: 14.82s (3373.8 samples/s)
```

Record the average wall-clock seconds per epoch for each configuration in your master tables.

7.3 Master Ablation Study Summary Tables (Mandatory Ladder)

Synthesize your empirical results into the two master tables below in your final report:

Table 7.3a: Architectural Configuration & Computational Complexity Profile (Report Deliverable)

Model Architecture	Conv Depth	Normalization & Regularization	Data Augmentation	Trainable Params (M)	FLOPs (MFLOPs)	Mean Epoch Time (s)	Inference Latency (ms)
1. Base LeNet-5	2	None	None	*[Record]*	*[Record]*	*[Record]*	*[Record]*
2. Modernized LeNet	2	None (ReLU + Overlapping MaxPool)	None	*[Record]*	*[Record]*	*[Record]*	*[Record]*
3. AlexNet (No Norm)	5	Dropout (\(p=0.5\))	None	*[Record]*	*[Record]*	*[Record]*	*[Record]*
4. AlexNet + LRN	5	LRN + Dropout (\(p=0.5\))	None	*[Record]*	*[Record]*	*[Record]*	*[Record]*
5. AlexNet + BatchNorm	5	`BatchNorm2d` + Dropout (\(p=0.5\))	None	*[Record]*	*[Record]*	*[Record]*	*[Record]*
6. AlexNet + BN + Aug	5	BatchNorm + Dropout (\(p=0.5\))	Pure PyTorch Aug	*[Record]*	*[Record]*	*[Record]*	*[Record]*
7. PyTorch Zoo AlexNet	5	PyTorch Zoo (`AdaptiveAvgPool2d`)	None	*[Record]*	*[Record]*	*[Record]*	*[Record]*

Table 7.3b: Empirical Predictive Performance Across Datasets (Report Deliverable)

Model Architecture	CIFAR-10 Val Acc (%)	CIFAR-10 Val Macro F_1 (%)	Tiny ImageNet Val Acc (%)	Tiny ImageNet Val Macro F_1 (%)
1. Base LeNet-5	*[Record]*	*[Record]*	*[Record]*	*[Record]*
2. Modernized LeNet	*[Record]*	*[Record]*	*[Record]*	*[Record]*
3. AlexNet (No Norm)	*[Record]*	*[Record]*	*[Record]*	*[Record]*
4. AlexNet + LRN	*[Record]*	*[Record]*	*[Record]*	*[Record]*
5. AlexNet + BatchNorm	*[Record]*	*[Record]*	*[Record]*	*[Record]*
6. AlexNet + BN + Aug	*[Record]*	*[Record]*	*[Record]*	*[Record]*
7. PyTorch Zoo AlexNet	*[Record]*	*[Record]*	*[Record]*	*[Record]*

Note

Reference Guidelines & Expected Empirical Scales:

- **Trainable Parameters:** Base LeNet $\sim 0.34\text{ M}$, Modernized LeNet $\sim 0.45\text{ M}$, AlexNet-64 $\sim 6.31\text{ M}$, PyTorch Zoo AlexNet $\sim 57.04\text{ M}$. Use these nominal orders of magnitude to verify that your tensor shapes and dense layer projections are properly defined.
- **Computational FLOPs:** Base LeNet $\sim 4.8\text{ MFLOPs}$, Modernized LeNet $\sim 8.2\text{ MFLOPs}$, AlexNet-64 $\sim 250\text{--}257\text{ MFLOPs}$, PyTorch Zoo AlexNet $\sim 310\text{--}315\text{ MFLOPs}$.
- **Empirical Variations:** Exact wall-clock timings depend on your host hardware (GPU model, PCIe bandwidth, or CPU threads), and validation metrics fluctuate stochastically across random weight seeds. In your report, focus on the expected progressive trends: modernized activations and pooling accelerate convergence; BatchNorm stabilizes optimization over LRN and unnormalized networks; data augmentation contracts the generalization gap Δ; and Tiny ImageNet produces lower absolute accuracy than CIFAR-10 due to increased class count ($K=200$) and higher task complexity.

Generate a multi-curve comparative plot using the provided visualization utility:

```
python -m utils.visualization --plot-ablation
```

Step 6: Post-Lab Analytical Report Guidelines

8.1 Required Report Sections

Your submitted report must be clear, concise, and structured with the following formal sections:

1. **Title & Header:** Course code (ELEC 475), Lab 2, your full name, student number, and partner information (if working in pairs).

2. **Architectural Specifications:** Detailed textual and mathematical descriptions of Base LeNet-5, Modernized LeNet, Full AlexNet-64, and the PyTorch Model Zoo AlexNet baseline. Include layer dimensions, receptive field calculations, and parameter breakdowns.

3. **Empirical Results & Figures:**

    - Embed the Master Ablation Study Summary Tables (Tables 7.3a & 7.3b, incorporating Accuracy, Macro F_1, Parameters, FLOPs, Epoch Times, and Latency).
    - Embed your Extended Ablation Exploration Table detailing your chosen custom experiments with Δ comparisons.
    - Include multi-seed evaluation results reported as $\text{Mean} \pm \text{Std}\%$.
    - Include side-by-side training and validation loss/accuracy curves.
    - Include the sample output from `demo_transforms.py` showing data types, shapes, and memory locations.
    - Include class balance and feature distribution figures from `utils/eda.py`.
    - Include the terminal output table from `python compare_models.py`.

4. **Technical Analysis & Discussion:**

    - analyze and discuss the empirical results
    - explain the differences in performance between the different model variations

---

Step 7: Submission Guidelines & Rubric

9.1 Submission Package Format & GitHub Repository Verification

Compress your work into a single archive named `<student_number>.zip` (or `<student1>_<student2>.zip` if working in pairs). Submit via OnQ before **October 22, 2026 at 23:59 ET**.

Your `.zip` archive must contain:

```
<student_number>/
├── models/
│   ├── __init__.py
│   ├── lenet.py
│   ├── lenet_modern.py
│   ├── alexnet.py
│   └── zoo.py                # PyTorch Model Zoo adapter
├── utils/
│   ├── __init__.py
│   ├── dataset.py
│   ├── eda.py
│   ├── transforms.py
│   ├── metrics.py            # Performance metrics (Accuracy, Confusion Matrix,
Macro/Micro/Weighted F1) & EarlyStopping controller
│   ├── visualization.py
│   ├── profiler.py           # FLOPs, MACs, Parameters & CUDA Timer profiler
│   └── seed.py               # Global seed management & statistical aggregation (Mean ± Std)
├── configs/
│   └── config.py
├── checkpoints/
│   ├── lenet_baseline.pth
│   ├── lenet_modern.pth
│   └── alexnet_bn.pth
├── plots/
│   ├── loss_lenet.png
│   ├── loss_lenet_modern.png
│   ├── loss_alexnet_bn.png
│   └── ablation_summary.png
├── demo_transforms.py
├── compare_models.py         # Multi-architecture FLOPs & complexity comparison harness
├── train.py
├── evaluate.py
├── github_repo.txt           # Text file with link to public/university GitHub repository
├── commands.txt              # Text file containing exact CLI terminal commands to run all
models
└── ELEC475_Lab2_Report.pdf   # Formal PDF Report
```

Caution

Do NOT include the `data/` folder in your `.zip` submission! Datasets are automatically downloaded via script and will unnecessarily inflate submission file size.

9.2 Lab 2 Overall Marking Rubric (7 Marks Total — 7% of Course Grade)

Assessment Item	Deliverable & Criteria	Marks
In-Lab Milestone	Evaluated in lab session: working LeNet, modernized LeNet, and TA checkoff.	**3.0**
Full AlexNet Architecture & Zoo Baseline	Correct 5-stage convolutional model in `models/alexnet.py`, FC layers with Dropout (\(p=0.5\)), and integration of PyTorch Model Zoo baseline in `models/zoo.py`.	1.0
Empirical Normalization & Extended Ablation Study	Experiments comparing No Norm vs. LRN vs. `BatchNorm2d` on AlexNet. Complete master ablation table (including Macro \(F_1\)) and extended ablation exploration menu (with \(\text{Mean} \pm \text{Std}\) multi-seed reporting) including parameters, FLOPs, and wall-clock training times measured via `Timer`.	1.0
PyTorch Data Augmentation	Implementation of RandomCrop, HorizontalFlip, ColorJitter in `transforms.py` and evaluation on CIFAR-10 and Tiny ImageNet.	0.5
Technical Analysis & Discussion	Correct and complete desciption and discussion of results.	1.0
Submission and Format	Clean code with `utils/seed.py` global seeding, valid `<student_number>.zip`, valid *.zip submission, exclusion of dataset caches, reproducible commands.	0.5
Post-Lab Subtotal		**4.0**
Total Lab 2 Grade		**7.0**